مبانی برنامه‌نویسی

مقدمه

امروزه کامپیوتر یکی از مهم‌ترین ابزارهای زندگی انسان است و در خانه، مدرسه، دانشگاه، شرکت‌ها، بانک‌ها، بیمارستان‌ها و بسیاری از مکان‌های دیگر مورد استفاده قرار می‌گیرد. برای استفاده صحیح از کامپیوتر لازم است با مفاهیم اولیه مانند سخت‌افزار، نرم‌افزار، سیستم‌عامل و برنامه‌نویسی آشنا شویم.

۱ .کامپیوتر از چه بخش‌هایی تشکیل شده است؟

کامپیوتر به طور کلی از دو بخش اصلی تشکیل شده است:

الف) سخت‌افزار(Hardware)

سخت‌افزار به تمام قسمت‌های فیزیکی و قابل لمس کامپیوتر گفته می‌شود

نمونه‌هایی از سخت‌افزار

- پردازنده (CPU)
- حافظه اصلی (RAM)
- هارد دیسک یا SSD
- مادربرد
- صفحه نمایش (Monitor)
- صفحه کلید (Keyboard)
- ماوس (Mouse)
- چاپگر (Printer)

وظیفه سخت‌افزار :اجرای دستوراتی است که نرم‌افزار به آن می‌دهد.

ب) نرم‌افزار(Software)

نرم‌افزار مجموعه‌ای از برنامه‌ها و دستورالعمل‌هایی است که به کامپیوتر می‌گوید چه کاری انجام دهد

نمونه‌هایی از نرم‌افزار

- Windows

- Linux
- Microsoft Word
- Google Chrome
- Photoshop
- بازی‌های کامپیوتری

بدون نرم‌افزار، سخت‌افزار هیچ کاربردی ندارد

۲ .مزایا و معایب کامپیوتر

مزایا

- سرعت بسیار بالا
- دقت زیاد
- ذخیره حجم زیادی از اطلاعات
- انجام محاسبات پیچیده
- خسته نشدن
- انجام کارهای تکراری بدون کاهش کیفیت
- ارتباط با اینترنت و جهان

معایب

- نداشتن خلاقیت
- نداشتن احساس
- تصمیم‌گیری فقط بر اساس برنامه
- وابستگی کامل به برق
- وابستگی به برنامه‌نویسی انسان
- احتمال خرابی سخت‌افزار
- خطر ویروس‌ها و حملات سایبری

۳ .کامپیوتر چه کارهایی نمی‌تواند انجام دهد؟

کامپیوتر بسیار قدرتمند است، اما محدودیت‌هایی نیز دارد

کامپیوتر:

- خلاق نیست
- احساس ندارد
- نمی‌تواند مانند انسان فکر کند
- بدون برنامه قادر به انجام هیچ کاری نیست
- قدرت درک واقعی ندارد
- نمی‌تواند به صورت مستقل تصمیم‌گیری منطقی مانند انسان انجام دهد (مگر در محدوده‌ای که برای آن برنامه‌ریزی شده باشد)

به عبارت دیگر:

کامپیوتر فقط همان کاری را انجام می‌دهد که انسان برای آن تعریف کرده است

۴ .برنامه‌نویسی چیست؟

برنامه‌نویسی (Programming) فرآیند نوشتن مجموعه‌ای از دستورها برای کامپیوتر است تا بتواند یک مسئله را حل کند یا کاری را انجام دهد

برنامه‌نویس با استفاده از زبان‌های برنامه‌نویسی، به کامپیوتر دستور می‌دهد که چگونه اطلاعات را پردازش کند

۵ .برنامه چیست؟

برنامه (Program) مجموعه‌ای از دستورها است که به ترتیب اجرا می‌شوند تا وظیفه مشخصی را انجام دهند

مثال:

- ماشین حساب
- بازی
- مرورگر اینترنت
- نرم‌افزار بانک

- برنامه مدیریت فروشگاه

۶ .مراحل ساخت یک برنامه کامپیوتری

ساخت یک نرم‌افزار معمولاً مراحل زیر را طی می‌کند:

۱ .تحلیل مسئله(Problem Analysis)

ابتدا مشخص می‌شود که برنامه قرار است چه مشکلی را حل کند

مثال:

ساخت برنامه مدیریت کتابخانه

۲ .طراحی(Design)

در این مرحله نحوه عملکرد برنامه طراحی می‌شود

- طراحی الگوریتم
- طراحی فلوچارت
- طراحی پایگاه داده
- طراحی رابط کاربری

۳ .برنامه‌نویسی(Coding)

در این مرحله برنامه‌نویس کدهای برنامه را می‌نویسد

۴ .ترجمه برنامه Compilation) یا(Interpretation

کد نوشته شده به زبان قابل فهم برای کامپیوتر تبدیل می‌شود

۵ .آزمایش(Testing)

خطاهای برنامه پیدا و اصلاح می‌شوند

۶ .اجرا(Execution)

برنامه توسط کاربر اجرا می‌شود

۷ .نگهداری و به‌روزرسانی(Maintenance)

پس از انتشار، برنامه به مرور زمان اصلاح و به‌روزرسانی می‌شود

۷ .زبان مشترک بین انسان و کامپیوتر چیست؟

انسان زبان طبیعی (فارسی، انگلیسی و ...) را می‌فهمد، اما کامپیوتر فقط زبان ماشین را متوجه می‌شود

برای برقراری ارتباط بین انسان و کامپیوتر از زبان‌های برنامه‌نویسی استفاده می‌شود

بنابراین:

زبان برنامه‌نویسی، زبان مشترک بین انسان و کامپیوتر است

۸ .زبان‌های برنامه‌نویسی چیست؟

زبان برنامه‌نویسی مجموعه‌ای از قوانین و دستورهاست که برنامه‌نویسان با استفاده از آن برنامه تولید می‌کنند

نمونه‌هایی از زبان‌های برنامه‌نویسی

- Python
- Java
- C
- C++
- C#
- JavaScript
- PHP
- Swift
- Kotlin
- Go
- Rust

هر زبان برای کاربردهای خاصی مناسب‌تر است

۹ .زبان ماشین چیست؟

پایین‌ترین سطح زبان برنامه‌نویسی، **زبان ماشین (Machine Language)** است

در این زبان، تمام دستورها فقط با دو عدد نوشته می‌شوند:

- 0
- 1

مثال:

10110011

00101010

11001001

این زبان مستقیماً توسط پردازنده اجرا می‌شود.

مزایای زبان ماشین

- سرعت اجرای بسیار بالا
- ارتباط مستقیم با پردازنده

معایب زبان ماشین

- بسیار دشوار برای انسان
- احتمال خطای زیاد
- خوانایی بسیار کم
- برنامه‌نویسی زمان‌بر

به همین دلیل امروزه بیشتر از زبان‌های سطح بالا مانند Python و Java استفاده می‌شود

۱۰ .سیستم‌عامل چیست؟

سیستم‌عامل Operating System یا OS مهم‌ترین نرم‌افزار یک کامپیوتر است که بین سخت‌افزار و برنامه‌های کاربردی ارتباط برقرار می‌کند.

وظایف سیستم‌عامل:

- مدیریت پردازنده (CPU)
- مدیریت حافظه (RAM)
- مدیریت فایل‌ها

- مدیریت دستگاه‌های ورودی و خروجی
- اجرای برنامه‌ها
- مدیریت کاربران
- تأمین امنیت سیستم

نمونه‌هایی از سیستم‌عامل

- Windows
- Linux
- macOS
- Android
- iOS

۱۱ .برنامه‌های کاربردی(Application Software)

برنامه‌های کاربردی نرم‌افزارهایی هستند که برای انجام کارهای خاص توسط کاربران طراحی شده‌اند

نمونه‌ها:

- Microsoft Word → تایپ متن
- Microsoft Excel → محاسبات و جدول
- PowerPoint → ساخت ارائه
- Photoshop → ویرایش تصویر
- Chrome → مرور اینترنت
- VLC → پخش فیلم
- Visual Studio Code → برنامه‌نویسی

۱۲ .تفاوت سیستم‌عامل و برنامه‌های کاربردی

برنامه کاربردی	سیستم‌عامل
یک کار خاص را انجام می‌دهد	مدیریت کل کامپیوتر را انجام می‌دهد
بدون آن نیز کامپیوتر کار می‌کند	بدون آن کامپیوتر قابل استفاده نیست
بعد از سیستم‌عامل اجرا می‌شود	اولین نرم‌افزاری است که اجرا می‌شود
مانندWord ، Excelو Photoshop	مانند Windows و Linux

تاریخچه برنامه‌نویسی :

از زمان اختراع اولین کامپیوترها، انسان همواره به دنبال راهی بوده است تا بتواند به راحتی با کامپیوتر ارتباط برقرار کند و به آن دستور دهد. در ابتدا این کار بسیار دشوار بود، زیرا کامپیوتر فقط زبان ماشین را متوجه می‌شد. با گذشت زمان، زبان‌های برنامه‌نویسی پیشرفته‌تری ایجاد شدند تا برنامه‌نویسی ساده‌تر، سریع‌تر و قابل فهم‌تر شود

به طور کلی، تاریخچه برنامه‌نویسی را می‌توان به سه نسل اصلی تقسیم کرد:

۱ .برنامه‌نویسی به زبان ماشین(Machine Language)

اولین زبان برنامه‌نویسی، زبان ماشین بود

این زبان تنها از دو رقم تشکیل شده است:

- 0
- 1

کامپیوتر تمامی دستورها را به صورت رشته‌ای از صفر و یک دریافت و اجرا می‌کند

نمونه

10110010

00101100

11100010

مزایا

- سریع‌ترین زبان برای اجرا
- مستقیماً توسط پردازنده اجرا می‌شود

- نیازی به ترجمه ندارد

معایب

- بسیار سخت و پیچیده
- احتمال خطای زیاد
- خواندن و نوشتن آن دشوار است
- هر پردازنده زبان ماشین مخصوص خود را دارد

به همین دلیل برنامه‌نویسی با زبان ماشین امروزه تقریباً انجام نمی‌شود

۲ .زبان اسمبلی(Assembly Language)

برای رفع مشکلات زبان ماشین، زبان اسمبلی معرفی شد

در این زبان، به جای صفر و یک از کلمات اختصاری (Mnemonic) استفاده می‌شود

مثال

MOV AX, 5

ADD AX, 3

SUB AX, 1

در این مثال:

- MOVیعنی انتقال اطلاعات
- ADDیعنی جمع
- SUBیعنی تفریق

نوشتن برنامه با اسمبلی نسبت به زبان ماشین بسیار ساده‌تر است

مزایای اسمبلی

- خوانایی بیشتر نسبت به زبان ماشین
- سرعت اجرای بالا
- کنترل مستقیم بر سخت‌افزار

- مناسب برای برنامه‌های سیستمی و درایورها

معایب اسمبلی

- یادگیری دشوار
- وابسته به نوع پردازنده
- برنامه‌های طولانی و پیچیده

۳ .زبان‌های برنامه‌نویسی سطح بالا(High-Level Languages)

با پیشرفت علم کامپیوتر، زبان‌هایی طراحی شدند که بسیار شبیه زبان انسان هستند

این زبان‌ها را زبان‌های سطح بالا می‌گویند

در این زبان‌ها دیگر نیازی نیست برنامه‌نویس با صفر و یک یا دستورهای اسمبلی کار کند

:مثال

```
x = 5
y = 10
print(x + y)
```

یا

```
#include <iostream>

int main()
{
   std::cout << "Hello World";
}
```

این کدها برای انسان قابل فهم هستند، اما کامپیوتر آن‌ها را مستقیماً متوجه نمی‌شود.

مترجم(Translator)

کامپیوتر فقط زبان ماشین را اجرا می‌کند

بنابراین برنامه‌ای که ما با زبان‌هایی مانندPython ، C++یا Java می‌نویسیم، باید ابتدا به زبان ماشین تبدیل شود

این کار توسط **مترجم (Translator)** انجام می‌شود

مترجم دو نوع اصلی دارد:

۱ .کامپایلر (Compiler)

تمام برنامه را یک‌جا به زبان ماشین ترجمه می‌کند

نمونه زبان‌ها:

- C
- C++
- Go
- Rust

۲ .مفسر (Interpreter)

برنامه را خط به خط ترجمه و اجرا می‌کند

نمونه زبان‌ها:

- Python
- JavaScript
- Ruby

روند اجرای یک برنامه

مراحل اجرای یک برنامه به صورت زیر است:

برنامه‌نویس

▼

نوشتن برنامه با زبان برنامه‌نویسی

▼

کامپایلر یا مفسر (مترجم)

▼

تبدیل به زبان ماشین (۰ و ۱)

▼

پردازنده(CPU)

▼

اجرای برنامه

بنابراین آنچه را که نیاز داریم با استفاده از یک زبان برنامه‌نویسی می‌نویسیم و سپس با استفاده از یک مترجم (کامپایلر یا مفسر)، آن را به زبان ماشین تبدیل می‌کنیم تا توسط پردازنده اجرا شود

زبان‌های برنامه‌نویسی سطح پایین(Low-Level Languages)

زبان‌های سطح پایین به سخت‌افزار بسیار نزدیک هستند.

نمونه‌ها:

- زبان ماشین (Machine Language)
- زبان اسمبلی (Assembly Language)

ویژگی‌ها

- سرعت اجرای بسیار بالا

- کنترل کامل روی سخت‌افزار
- برنامه‌نویسی دشوار
- وابسته به نوع پردازنده

زبان‌های برنامه‌نویسی سطح بالا(High-Level Languages)

این زبان‌ها به زبان انسان نزدیک‌تر هستند و یادگیری آن‌ها آسان‌تر است.

نمونه‌ها:

- Python
- Java
- C#
- C++
- JavaScript
- PHP
- Swift
- Kotlin
- Go
- Rust
- Dart

ویژگی‌ها

- خوانایی بالا
- یادگیری آسان
- توسعه سریع نرم‌افزار
- قابلیت اجرا روی سیستم‌های مختلف (در بسیاری از زبان‌ها)
- نگهداری و توسعه آسان‌تر

مقایسه زبان‌های سطح پایین و سطح بالا

ویژگی	زبان‌های سطح پایین	زبان‌های سطح بالا
نزدیکی به سخت‌افزار	بسیار زیاد	کم
خوانایی	بسیار کم	بسیار زیاد
یادگیری	دشوار	آسان
سرعت توسعه	کم	زیاد
وابستگی به پردازنده	زیاد	کم
نیاز به مترجم	اسمبلی نیاز دارد، زبان ماشین خیر	بله
نمونه‌ها	Assembly ،Machine Language	PHP ،JavaScript ،C# ،Java ،C++ ،Python

زبان برنامه‌نویسی	نمونه کد
Machine Code	10111000 00000010
Assembly	asm\nMOV AX, 2\nADD AX, 2\n
C	c\n#include <stdio.h>\nint main() {\n printf(\"%d\", 2 + 2);\n return 0;\n}\n
C++	cpp\n#include <iostream>\nusing namespace std;\nint main() {\n cout << 2 + 2;\n return 0;\n}\n
Java	java\npublic class Main {\n public static void main(String[] args) {\n System.out.println(2 + 2);\n }\n}\n
C#	csharp\nusing System;\nclass Program {\n static void Main() {\n Console.WriteLine(2 + 2);\n }\n}\n
Python	python\nprint(2 + 2)\n
JavaScript	javascript\nconsole.log(2 + 2);\n
PHP	php\n<?php\necho 2 + 2;\n?>\n
Go	go\npackage main\nimport \"fmt\"\nfunc main() {\n fmt.Println(2 + 2)\n}\n
Rust	rust\nfn main() {\n println!(\"{}\", 2 + 2);\n}\n
Swift	swift\nprint(2 + 2)\n
Kotlin	kotlin\nfun main() {\n println(2 + 2)\n}\n
Dart	dart\nvoid main() {\n print(2 + 2);\n}\n
Ruby	ruby\nputs 2 + 2\n
R	r\nprint(2 + 2)\n
MATLAB	matlab\ndisp(2 + 2)\n
Julia	julia\nprintln(2 + 2)\n
Pascal	pascal\nprogram Test;\nbegin\n writeln(2 + 2);\nend.\n
Visual Basic .NET	vb\nModule Program\n Sub Main()\n Console.WriteLine(2 + 2)\n End Sub\nEnd Module\n
Perl	perl\nprint 2 + 2;\n
Lua	lua\nprint(2 + 2)\n
COBOL	cobol\nCOMPUTE RESULT = 2 + 2.\nDISPLAY RESULT.\n
Fortran	fortran\nprogram main\nprint *, 2 + 2\nend program main\n

اجزای تشکیل‌دهنده برنامه‌ها و انواع دستورها در برنامه‌نویسی

هر نرم‌افزار یا برنامه کامپیوتری از مجموعه‌ای از دستورها (Instructions) تشکیل شده است. این دستورها به ترتیب اجرا می‌شوند و به کامپیوتر می‌گویند که چه عملیاتی را انجام دهد. هر زبان برنامه‌نویسی، مجموعه‌ای از دستورها و قواعد (Syntax) مخصوص به خود را دارد، اما اکثر زبان‌ها از انواع مشابهی از دستورها پشتیبانی می‌کنند.

۱ .برنامه از چه چیزی تشکیل شده است؟

هر برنامه از بخش‌های زیر تشکیل می‌شود:

- متغیرها (Variables)
- ثابت‌ها (Constants)
- عملگرها (Operators)
- توابع (Functions)
- کلاس‌ها (در زبان‌های شیءگرا)
- دستورها Instructions) یا (Commands

مهم‌ترین بخش هر برنامه، دستورها هستند؛ زیرا مشخص می‌کنند برنامه چه کاری انجام دهد.

۲ .انواع دستورها در برنامه‌نویسی

الف) دستورات فراخوانی عملیات(Operation Call)

این دستورها یک عملیات یا تابع را اجرا می‌کنند

مثال درPython

```
print("Hello World")
sqrt(25)
input("Enter your name: ")
```

ب) دستورات تعریف(Definition)

برای تعریف متغیر، تابع، کلاس یا ساختارهای دیگر استفاده می‌شوند

تعریف متغیر

```
x = 10
```

تعریف تابع

```
def add(a, b):
return a + b
```

تعریف کلاس

```
class Student:
pass
```

ج) دستورات انتساب(Assignment)

برای قرار دادن یک مقدار داخل متغیر استفاده می‌شوند

```
age = 25
name = "Ali"
total = x + y
```

د) دستورات بازگشت(Return)

این دستور مقدار خروجی یک تابع را مشخص می‌کند

```
def square(x):
return x * x
```

بدون دستور **return**، تابع مقداری به برنامه برنمی‌گرداند

هـ) دستورات پرش(Go To)

این دستور باعث انتقال اجرای برنامه به نقطه‌ای دیگر می‌شود

در زبان‌های قدیمی مانند:BASIC

```
GOTO 100
```

در زبان‌های مدرن مانندPython ، دستور **goto**وجود ندارد؛ زیرا استفاده زیاد از آن باعث کاهش خوانایی برنامه می‌شود. امروزه به جای آن از حلقه‌ها، توابع و ساختارهای کنترلی استفاده می‌شود

و) دستورات ترکیبی(Compound Statements)

این دستورات از چندین دستور ساده تشکیل شده‌اند

مثال:

```
if score >= 10:
   print("Pass")
   print("Congratulations")
```

یا

```
for i in range(5):
   print(i)
```

ز) دستورات تصمیم‌گیری(Decision Statements)

این دستورات باعث می‌شوند برنامه بر اساس یک شرط تصمیم بگیرد

if

```
age = 20
if age >= 18:
   print("Adult")
```

if-else

```
if score >= 10:
   print("Pass")
else:
   print("Fail")
```

if-elif-else

```
score = 18

if score >= 18:
```

```
    print("Excellent")
elif score >= 15:
    print("Good")
else:
    print("Normal")
```

ح) دستورات تکرار(Loop Statements)

گاهی لازم است یک دستور چندین بار اجرا شود

برای این کار از حلقه‌ها استفاده می‌کنیم

حلقهFor

```
for i in range(5):
    print(i)
```

خروجی:

```
0
1
2
3
4
```

حلقهWhile

```
i = 0
while i < 5:
    print(i)
    i += 1
```

تفاوت For وWhile

While	For
تعداد تکرار ممکن است مشخص نباشد.	تعداد تکرار معمولاً مشخص است.
انعطاف‌پذیرتر	خوانایی بیشتر
مناسب شرط‌های پیچیده	مناسب پیمایش لیست‌ها

۳ .انواع خطاها در برنامه‌نویسی

هنگام نوشتن برنامه ممکن است خطاهای مختلفی رخ دهد

الف) خطای نحوی(Syntax Error)

این خطا زمانی رخ می‌دهد که قوانین نگارشی زبان رعایت نشده باشد

مثال:

```
if x > 5
print(x)
```

فراموش شدن :باعث ایجاد Syntax Error می‌شود

ب) خطای زمان ترجمه(Compile-Time Error)

این خطا در زبان‌های کامپایلری مانند C و C++ هنگام کامپایل برنامه رخ می‌دهد

مثال در:C

```
printf("Hello")
```

اگر ;فراموش شود:

```
printf("Hello")
```

کامپایلر خطا می‌دهد و برنامه ساخته نمی‌شود

ج) خطای زمان اجرا(Run-Time Error)

این خطا پس از اجرای برنامه رخ می‌دهد

مثال:

```
x = 10
y = 0

print(x / y)
```

خروجی:

ZeroDivisionError

نمونه‌های دیگر:

- تقسیم بر صفر
- باز کردن فایل وجود نداشته
- دسترسی به اندیس نامعتبر
- کمبود حافظه

د) خطای منطقی(Logical Error)

در این حالت برنامه بدون خطا اجرا می‌شود، اما نتیجه اشتباه است

مثال:

```
length = 10
width = 5

area = length + width
```

برنامه اجرا می‌شود اما فرمول مساحت اشتباه است. باید از ضرب استفاده شود

```
area = length * width
```

این نوع خطا معمولاً سخت‌ترین خطا برای پیدا کردن است

۴ .مدیریت خطاها در Python

در پایتون از ساختار **try-except** برای مدیریت خطاها استفاده می‌شود

```
try:
    number = 10 / 0
except ZeroDivisionError:
    print("تقسیم بر صفر مجاز نیست.")
```

همچنین می‌توان از بخش‌های else و finally استفاده کرد

```
try:
    x = int(input())
except ValueError:
    print("ورودی معتبر نیست.")
else:
    print("ورودی صحیح است.")
finally:
    print("پایان برنامه")
```

شرایط لازم برای کامل بودن یک زبان برنامه‌نویسی

یک زبان برنامه‌نویسی برای اینکه بتواند هر نوع الگوریتمی را پیاده‌سازی کند، باید حداقل سه قابلیت اساسی داشته باشد:

۱ .اجرای ترتیبی(Sequence)

دستورها باید بتوانند به صورت خط به خط و به ترتیب اجرا شوند

مثال:

```
a = 5
b = 10
print(a + b)
```

ابتدا مقدار a، سپس bو در پایان نتیجه چاپ می‌شود.

۲ .تصمیم‌گیری و انشعاب(Selection)

زبان باید بتواند بر اساس یک شرط، مسیر اجرای برنامه را تغییر دهد

مثال:

```
if temperature > 30:
   print("هوا گرم است.")
else:
   print("هوا معتدل است.")
```

۳ .تکرار(Iteration)

زبان باید امکان اجرای چندباره یک یا چند دستور را فراهم کند

مثال:

```
for i in range(10):
   print(i)
```

یا

```
while True:
   print("Hello")
```

انواع داده و ساختارهای مدیریت و ذخیره سازی اطلاعات :

معرفی انواع داده‌ها در زبان‌های برنامه‌نویسی

در زبان‌های برنامه‌نویسی مختلف، داده‌ها به انواع متعددی تقسیم می‌شوند که هر یک ویژگی‌ها و کاربردهای خاص خود را دارند. تفکیک و شناخت این انواع داده‌ای برای طراحی و توسعه برنامه‌های مؤثر اهمیت فراوانی دارد

انواع داده عددی

۱. **اعداد صحیح (Integer)**: این اعداد می‌توانند مقادیر مثبت، منفی یا صفر را بپذیرند و براساس تعداد بیت‌هایی که برای ذخیره‌سازی نیاز دارند با مشخصه‌هایی نظیر int8، int16، int32 و غیره معرفی می‌شوند. حالت‌های بدون علامت (unsigned) فقط مقادیر نامنفی را شامل می‌شوند، در حالی که حالت‌های دارای علامت (signed) شامل اعداد مثبت و منفی هستند.

۲. **اعداد اعشاری (Floating Point)**: این نوع داده‌ها برای نمایش اعداد با ممیز شناور به کار می‌روند. Floating Point به این معناست که ممیز عدد می‌تواند در موقعیت‌های مختلف قرار گیرد تا محدوده وسیع‌تری از اعداد را نمایش دهد. این نوع داده‌ها به صورت float، double و ... در زبان‌هایی مانند C، C++ و Python تعریف می‌شوند.

۳. **اعداد مختلط (Complex)**: در برخی زبان‌ها همچون MATLAB یا Python امکان تعریف و کار با اعداد مختلط وجود دارد؛ با این حال بسیاری از زبان‌ها پشتیبانی مستقیمی برای این نوع داده ندارند یا آن را به صورت تقریبی پیاده‌سازی می‌کنند.

انواع داده منطقی (Boolean):

داده منطقی فقط می‌تواند یکی از دو مقدار "درست" (true) یا "نادرست" (false) را بپذیرد. این نوع داده در بسیاری از زبان‌های برنامه‌نویسی وجود دارد و گاهی به مقادیر عددی صفر و یک نیز تبدیل می‌شود. برخی زبان‌ها مانند C در نسخه اولیه نوع داده منطقی قابل شناسایی نداشتند، اما نسخه‌های جدیدتر و دیگر زبان‌ها این قابلیت را اضافه کرده‌اند

انواع داده متنی:

داده‌های متنی شامل کاراکتر (char) و رشته (string) هستند:

۱-کاراکتر: نشان‌دهنده یک نماد منفرد است، مانند "a"، "b" یا هر علامت دیگر. در زبان‌های برنامه‌نویسی مانند C، C++، و Java نوع داده‌ای به نام char تعریف شده است.

۲-رشته: شامل مجموعه‌ای از کاراکترهاست که معنای خاصی در نرم‌افزارها دارند. در برخی زبان‌ها مانند C، رشته به صورت آرایه‌ای از کاراکترها ذخیره می‌شود؛ در مقابل زبان‌هایی مانند C# رشته را به عنوان نوع داده مستقل (string) تعریف می‌کنند

تبدیل انواع داده (Type Casting)

در بسیاری از زبان‌های برنامه‌نویسی، تبدیل انواع داده امکان‌پذیر است. بسته به نوع تبدیل، این فرآیند یا به صورت ضمنی (implicit) انجام می‌شود، یا به طور صریح (explicit) و با اعلام مستقیم برنامه‌نویس:

- **تبدیل ضمنی (Implicit Casting)**: هنگامی که نوع داده کوچکتر (مثلاً uint8) به نوع داده بزرگتر (مثلاً uint16) تبدیل می‌شود و احتمال از دست رفتن داده وجود ندارد، این تبدیل معمولاً به صورت خودکار انجام می‌شود.

- **تبدیل صریح (Explicit Casting)**: هنگامی که نوع داده بزرگ‌تر یا دقت بالاتر به نوعی کوچکتر تبدیل می‌شود (که ممکن است باعث از دست رفتن داده شود)، معمولاً این تبدیل نیاز به اعلام مستقیم توسط برنامه‌نویس دارد.

حساسیت زبان‌های برنامه‌نویسی به نوع داده

زبان‌های برنامه‌نویسی از نظر نحوه مدیریت نوع داده به دو دسته اصلی تقسیم می‌شوند:

- **زبان‌های با حساسیت بالا به نوع داده (Type-Safe)**: زبان‌هایی مانند C، C++، Java و C# نیازمند تعریف صریح نوع داده برای متغیرها هستند؛ بنابراین عملیات ناسازگار با نوع داده خطا تولید می‌کند. این ویژگی ایمنی و سلامت برنامه را افزایش می‌دهد.
- **زبان‌های با حساسیت پایین به نوع داده:** زبان‌هایی مانند PHP، JavaScript و MATLAB در تعیین و استفاده از نوع داده‌ها انعطاف بیشتری داشته و تغییر نوع داده در طول اجرا امکان‌پذیر است. این زبان‌ها اجرای سریع‌تر و توسعه آسان‌تر را برای برنامه‌های ساده فراهم می‌کنند اما ممکن است سبب بروز خطاهای ناخواسته در برنامه شوند.

معرفی ساختارهای داده‌ای ساده و پیچیده

انواع داده‌ای ساده مانند عددی، منطقی و متنی، پایه و اساس برنامه‌نویسی را تشکیل می‌دهند. برای مدیریت داده‌های پیچیده‌تر، ساختارهای زیر مورد استفاده قرار می‌گیرند:

۱. **متغیر (Variable)**:

متغیرها ظرف‌هایی برای نگهداری داده‌های مختلف هستند و مقدار آن‌ها در طول اجرای برنامه می‌تواند تغییر کند متغیرها معمولاً نوع داده مشخص دارند و ممکن است مقدار اولیه نداشته یا از کاربر، تغییریابند یا ثابت باشند

۲. **تابع (Function) و زیر برنامه (Subroutine)**:

توابع بلوک‌هایی از کد هستند که مجموعه‌ای از عملیات را با ورودی‌های مشخص انجام می‌دهند و معمولاً خروجی می‌دهند. استفاده از توابع موجب کدنویسی ساخت‌یافته، افزایش خوانایی و بازاستفاده می‌شود. توابع می‌توانند داده‌هایی از نوع متغیرها یا حتی توابع دیگر را به عنوان ورودی بپذیرند.

۳. **آرایه (Array)**:

آرایه مجموعه‌ای از عناصر با نوع داده یکسان است که به آن‌ها از طریق اندیس عددی دسترسی پیدا می‌شود. آرایه‌ها مدیریت داده‌های حجیم را تسهیل می‌کنند. در زبان‌های مختلف آرایه‌ها می‌توانند از اندیس ۰ یا ۱ شروع شوند.

٤. **ساختار (Structure)**:

ساختارها مجموعه‌ای از داده‌ها با نوع‌های داده‌ای متفاوت را تحت یک نام مشترک سامان‌دهی می‌کنند. برای مثال اطلاعات یک دانشجو (نام، نام خانوادگی، شماره دانشجویی، نمرات و ...) می‌تواند به عنوان یک ساختار تعریف شود. اندیس‌ها در ساختارها معمولاً نمادین بوده و با نام از هم متمایز می‌شوند.

۵. **ساختارهای ترکیبی و پیشرفته:**

افزون بر آرایه و ساختار، انواع پیشرفته‌تری نظیر سل آرِی (cell array) یا جَگد آرِی (jagged array) وجود دارند که ترکیبی از ویژگی‌های آرایه و ساختار را ارائه می‌دهند.

مفهوم کلاس و شیء (Object & Class)

کلاس، نوع داده‌ای پیشرفته است که ترکیبی از داده‌ها (ویژگی‌ها یا پراپرتی‌ها) و عملیات (متدها یا توابع) را در برمی‌گیرد. با استفاده از کلاس‌ها می‌توان مفاهیم دنیای واقعی را با ویژگی‌ها و رفتارهای مشخص در برنامه‌نویسی مدل‌سازی کرد. کلاس‌های تعریف‌شده برای تولید نمونه‌هایی به نام "شیء" یا "ابجکت" مورد استفاده قرار می‌گیرند.

ویژگی‌های کلاس و شیء عبارتند از: - امکان تعریف ویژگی‌های فقط‌خواندنی یا قابل‌تغییر (مانند حداکثر سرعت خودرو یا میزان سوخت). - وجود متدها یا عملیات (مانند شروع به حرکت یا توقف خودرو). - ارتباط بین کلاس‌ها به صورت سلسله‌مراتبی (وراثت)، به صورتی که کلاس‌های فرزند می‌توانند ویژگی‌ها و رفتارهای کلاس والد را به ارث ببرند. - هر نمونه (شیء) خصوصیات منحصر به فرد خود را دارد اما رفتار و ساختار اصلی را از کلاس دریافت کرده است.

برنامه‌نویسی شی‌گرا (Object Oriented Programming - OOP)

برنامه‌نویسی شی‌گرا یک رویکرد مدرن در توسعه نرم‌افزار است که بر ایجاد اشیاء (ابجکت‌ها) بر اساس کلاس‌ها تمرکز دارد. این روش مزایای متعددی از جمله بازاستفاده‌پذیری، نگهداری آسان، توسعه‌پذیری و سازمان‌دهی بهتر کد را فراهم می‌کند. از مهم‌ترین زبان‌های برنامه‌نویسی شی‌گرا می‌توان به C++، C# و Java اشاره کرد.

ساختمان پایه‌ای برنامه‌نویسی شی‌گرا عمدتاً بر مبنای کلاس و شیء است و زبان‌هایی که این پارادایم را دنبال می‌کنند، معمولاً برای هر برنامه حداقل یک کلاس و یک متد کلیدی (مانند main) الزامی می‌دانند.

سیر تکاملی زبان‌های برنامه‌نویسی

زبان‌های برنامه‌نویسی به تدریج از زبان‌های ساده و خطمحور مانند BASIC و COBOL با حداقل نیاز (مانند وجود یک دستور یا کامند) به زبان‌های ساخت‌یافته (مانند C و Pascal که وجود یک تابع الزامی است) و سپس به زبان‌های شی‌گرا (مانند Java و C# که وجود حداقل یک کلاس الزامی است) تکامل یافته‌اند.

توسعه قابلیت‌هایی همچون ساختار، آرایه، کلاس و شیء موجب افزایش پیچیدگی و قدرت برنامه‌سازی شده و امکان توسعه نرم‌افزارهای بزرگ، پویا و چندمنظوره را فراهم آورده است.